

Fiap_Pos_9AIDT_Fase_01_API

API em Python (Flask + Flasgger) que embarca um modelo XGBoost para predição de violência sexual a partir dos campos brutos de uma notificação de violência.

Estrutura do projeto

```
.
├── app.py                # Aplicação Flask com endpoint /predict e documentação
├── Swagger
├── predict.py            # Pré-processamento e lógica de predição
├── requirements.txt      # Dependências Python
└── model/                # Arquivos do modelo (não versionados)
    ├── xgb_viol_sexu.joblib
    ├── imputer.joblib
    └── feature_columns.joblib
```

Pré-requisitos

- Python 3.14+
- Arquivos de modelo em `model/` (gerados pelo pipeline de treino):
 - `xgb_viol_sexu.joblib`
 - `imputer.joblib`
 - `feature_columns.joblib`

Instalação

```
pip install -r requirements.txt
```

Execução

```
python app.py
```

A API estará disponível em <http://localhost:5000>.

A documentação Swagger (Flasgger) estará em <http://localhost:5000/docs/>.

Endpoint

POST `/predict`

Recebe um JSON com os campos brutos da notificação e retorna a predição do modelo.

Exemplo de requisição:

```
{
  "IDADE_ANOS": 23,
  "VIOL_FISIC": 1,
  "VIOL_PSICO": 1,
  "VIOL_TORT": 2,
  "VIOL_FINAN": 2,
  "VIOL_NEGLI": 2,
  "VIOL_INFAN": 2,
  "VIOL_LEGAL": 2,
  "VIOL_OUTR": 2,
  "LES_AUTOP": 2,
  "AUTOR_ALCO": 1,
  "OUT_VEZES": 1,
  "CS_GESTANT": 5,
  "CS_RACA": 4,
  "CS_ESCOL_N": 3,
  "LOCAL_OCOR": 1,
  "AUTOR_SEXO": 1
}
```

Exemplo de resposta:

```
{
  "probabilidade_viol_sexual": 0.8523,
  "classificacao": "violencia_sexual",
  "alerta": true
}
```

Campos de entrada

Campo	Tipo	Valores aceitos
IDADE_ANOS	Numérico	0–125
VIOL_FISIC, VIOL_PSICO, VIOL_TORT, VIOL_FINAN, VIOL_NEGLI, VIOL_INFAN, VIOL_LEGAL, VIOL_OUTR	Binário	1=Sim · 2=Não · 9/ausente=Ignorado
LES_AUTOP	Binário	1=Sim · 2=Não · 9/ausente=Ignorado
AUTOR_ALCO	Binário	1=Sim · 2=Não · 9/ausente=Ignorado
OUT_VEZES	Binário	1=Sim · 2=Não · 9/ausente=Ignorado

Campo	Tipo	Valores aceitos
CS_GESTANT	Categórico	1=1º tri · 2=2º tri · 3=3º tri · 4=Idade gestacional ignorada · 5=Não gestante · 6=Não se aplica · 8=Não informado
CS_RACA	Categórico	1=Branca · 2=Preta · 3=Amarela · 4=Parda · 5=Indígena
CS_ESCOL_N	Categórico	0=Sem escol. · 1=Fund. I incomp. · 2=Fund. I comp. · 3=Fund. II incomp. · 4=Fund. II comp. · 5=Médio incomp. · 6=Médio comp. · 7=Superior incomp. · 8=Superior comp. · 10=Não informado
LOCAL_OCOR	Categórico	1=Residência · 2=Hab. coletiva · 3=Escola · 4=Esporte · 5=Bar/Boate · 6=Via pública · 7=Comércio · 8=Indústria
AUTOR_SEXO	Categórico	1=Masculino · 2=Feminino · 3=Ambos · 4=Ignorado

Campos de saída

Campo	Tipo	Significado
probabilidade_viol_sexual	float (0–1)	Probabilidade estimada de violência sexual
classificacao	string	"violencia_sexual" ou "sem_violencia_sexual"
alerta	bool	true se prob $\geq$ 0.5 (threshold padrão)

Instalação via Docker Compose

Pré-requisitos

- [Docker](#) instalado (versão 20.10+)
- [Docker Compose](#) instalado (versão 2.0+ ou plugin `docker compose`)
- Arquivos de modelo presentes na pasta `model/` do projeto:
 - `xgb_viol_sexu.joblib`
 - `imputer.joblib`
 - `feature_columns.joblib`

Passo a passo

1. Clone o repositório

```
git clone https://github.com/Etyonamine/Fiap_Pos_9AIDT_Fase_01_API.git
cd Fiap_Pos_9AIDT_Fase_01_API
```

2. Adicione os arquivos de modelo

Copie os arquivos do modelo treinado para a pasta **model/** na raiz do projeto:

```
model/  
├── xgb_viol_sexu.joblib  
├── imputer.joblib  
└── feature_columns.joblib
```

Observação: Caso não existam, deve ser copiado do repositório do ML pasta model no endereço https://github.com/Etyonamine/Fiap_Pos_9AIDT_Fase_01_ML

3. Construa a imagem e suba o container

```
docker compose build --no-cache
```

Na primeira execução o Docker irá baixar a imagem base (**python:3.12-slim**) e instalar as dependências. As execuções seguintes serão mais rápidas pois o cache de camadas é reutilizado.

*Observação: Caso já exista a imagem no docker deve ser apagado

4. Verifique se o serviço está em execução

```
docker compose ps
```

A saída esperada mostra o container **fiap_fase01_api** com status **running** e a porta **5000** exposta.

5. Acesse a API

Recurso	URL
Endpoint de predição	http://localhost:5000/predict
Documentação Swagger	http://localhost:5000/docs/

6. (Opcional) Executar em segundo plano

Para rodar o serviço em modo *detached* (sem bloquear o terminal):

```
docker compose up --build -d
```

7. Parar o serviço

```
docker compose down
```

Para parar e remover também os volumes criados:

```
docker compose down -v
```